

Aspera HSTS Containers

2026/10/07

Contents

1	Prerequisites	2
2	Architecture	3
2.1	Containers	3
2.2	Volumes	4
2.3	Logging	4
3	Quickstart	5
4	Rake tasks	6
5	Environment variables	7
6	Node API: access keys or docroot	8
7	License	9

 Important

Experimental as a sample, not for production.

Build and run IBM Aspera High-Speed Transfer Server (HSTS) with Docker Compose: one HSTS container (Node API + SSH/FASP transfers) and one Redis container.

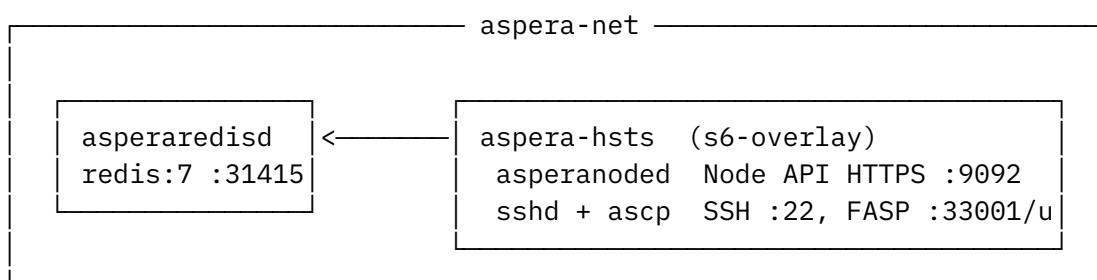
Chapter 1

Prerequisites

- Docker $\geq$ 24 with Compose v2
- Ruby $\geq$ 3.0 with `rake`, and the `rpm` command (macOS: `brew install rpm`) — used to read the RPM version and architecture
- An Aspera HSTS RPM placed in `private/` (exactly one)
- An Aspera license: license file or ALEE entitlement (see [License](#))

Chapter 2

Architecture



2.1 Containers

Container	Process(es)	Port(s)	Description
<code>asperaredis</code>	<code>redis-server</code> (stock <code>redis:7-alpine</code>)	31415/tcp (internal)	Node API database
<code>aspera-hsts</code>	<code>asperanoded</code> , <code>sshd</code> (+ <code>ascp</code> per transfer)	9092/tcp, 22/tcp, 33001/udp	Node API (HTTPS), SSH/FASP transfers

Inside `aspera-hsts`, `s6-overlay` runs these services ([rootfs/etc/s6-overlay/s6-rc.d](#)):

Service	Type	Role
<code>syslog-bridge</code>	longrun	Copies syslog messages from <code>/dev/log</code> to the container's stdout
<code>init-aspera-hsts</code>	oneshot	Prepares volumes and configuration (see below); the container stops if it fails
<code>asperanoded</code>	longrun	Node API, as <code>asperadaemon</code>
<code>sshd</code>	longrun	SSH for <code>ascp</code> , key authentication only, user <code>xfer</code> with <code>aspsell</code>

At each start, `init-aspera-hsts`:

- populates empty volumes from the image defaults and removes stale PID files
- points `asperanoded` to the Redis container (`db_host`, `db_port`)
- generates, on first start, the Node API TLS key pair and the SSH host keys (stored in the `etc` volume)
- writes `XFER_AUTHORIZED_KEYS` to `xfer`'s `authorized_keys` and configures `xfer`'s storage (see [Node API: access keys or docroot](#))
- installs the license (if provided)
- creates or updates the Node API user `NODE_USER`, mapped to `xfer`

2.2 Volumes

Volume	Mounted in	Content
aspera-etc	aspera-hsts: /opt/aspera/etc	aspera.conf, license, TLS key pair, SSH host keys
aspera-var	aspera-hsts: /opt/aspera/var	Runtime state, logs, token keys
redis-data	asperaredisd: /data	Redis database (Node API users, transfer history)

Transferred files are stored in /home/xfer by default, which is not a volume.

2.3 Logging

Aspera processes log via syslog. The syslog-bridge service listens on /dev/log and writes one line per message to stdout, so logs are available with docker compose logs or any log driver (Loki, CloudWatch, Splunk...).

Chapter 3

Quickstart

```
# 1. Create the configuration
cp .env.example .env
# Edit .env: NODE_PASS, XFER_AUTHORIZED_KEYS, license...

# 2. Build the image
rake build

# 3. Start
docker compose up -d

# 4. Check
docker compose ps
docker compose logs -f
curl -k -u "nodeuser:<NODE_PASS>" https://localhost:9092/info
```

Chapter 4

Rake tasks

```
rake build          # Build the aspera-hsts image (default task), for the RPM's architecture
rake push          # Push the image to REGISTRY
rake compose:up     # docker compose up -d
rake compose:down   # docker compose down
rake compose:logs   # docker compose logs -f
rake pdf           # Generate README.pdf from README.md
```

`rake pdf` uses `gfm2pdf-toolchain`: set `GFM2PDF_DIR` to its folder (default: `../gfm2pdf-toolchain`, next to this repository), and `GFM2PDF_ENGINE` to select the PDF engine (`latex` or `typst`).

Chapter 5

Environment variables

See `.env.example`.

Variable	Default	Description
REGISTRY	localhost	Docker registry
TAG	RPM version (rake), latest (compose)	Image tag
NODE_USER	nodeuser	Node API user
NODE_PASS	(required)	Node API password: 16-92 ASCII characters, at least 2 lowercase, 2 uppercase, 2 digits
NODE_PORT	9092	Node API HTTPS port (host)
ASCP_SSH_PORT	33001	SSH port (host)
FASP_UDP_PORT	33001	FASP UDP port (host)
XFER_AUTHORIZED_KEYS	(empty)	SSH public key(s) for xfer
XFER_DOCROOT	(empty)	Docroot of xfer: Node API Gen3 mode
XFER_RESTRICTION	file:///home/xfer/*\ /home/xfer/*	File restriction(s) of xfer, separated by \ , when XFER_DOCROOT is empty: access key mode
ASPERA_LICENSE	(empty)	Content of a license file
ASPERA_CUSTOMER_ID, ASPERA_ENTITLEMENT_ID	(empty)	ALEE entitlement, registered with alee-admin

Chapter 6

Node API: access keys or docroot

HSTS supports two ways to give the Node API access to the storage of the transfer user `xfer`, set by the init at each start:

	Access keys (default)	Docroot
Setting	<code>XFER_DOCROOT</code> empty: no docroot, file restriction(s) <code>XFER_RESTRICTION</code>	<code>XFER_DOCROOT</code> set
SSH transfers (<code>ascp</code>)	Absolute paths matching a restriction, e.g. <code>xfer@host:/home/xfer/</code>	Paths relative to the docroot, e.g. <code>xfer@host:/</code>
Access keys (Gen4, used by Aspera on Cloud, Faspex 5, <code>ascli node access_keys</code>)	Yes: storage paths must match a restriction	No: docroot must be disabled in <code>aspera.conf</code> to create <code>access_keys</code>
File operations by the Node user (Gen3: <code>/files/browse</code> , <code>ascli node browse</code>)	No: Invalid doc root path	Yes

Restrictions are matched against URLs for access key storage (`file:///home/xfer/*`) and against plain paths for SSH transfers (`/home/xfer/*`): the default sets both.

For example, with `ascli`:

```
ascli node --url=https://localhost:9092 --username=nodeuser --password=... \
  access_keys create @: id=my_ak secret=my_secret storage.type=local storage.path=/home/xfer
ascli node --url=https://localhost:9092 --username=my_ak --password=my_secret access_keys do self
↵ browse /
```

Chapter 7

License

Without a license, `asperanoded` answers every request (including `/ping`, so the healthcheck fails) with `Invalid server configuration: License issue`, and transfers are refused. Provide one of:

- `ASPERA_LICENSE`: content of the license file, written to `/opt/aspera/etc/aspera-license` at each start
- `ASPERA_CUSTOMER_ID` and `ASPERA_ENTITLEMENT_ID`: registered with `alee-admin register` at each start
- or copy a license file into the `etc` volume: `docker compose cp aspera-license aspera-hsts:/opt/aspera/etc/aspera-license`

End of document